

Quality Gate Challenge

Mise en place et utilisation de Sonar

PALERMO Enzo, SOLANA-DUPLAA Léo

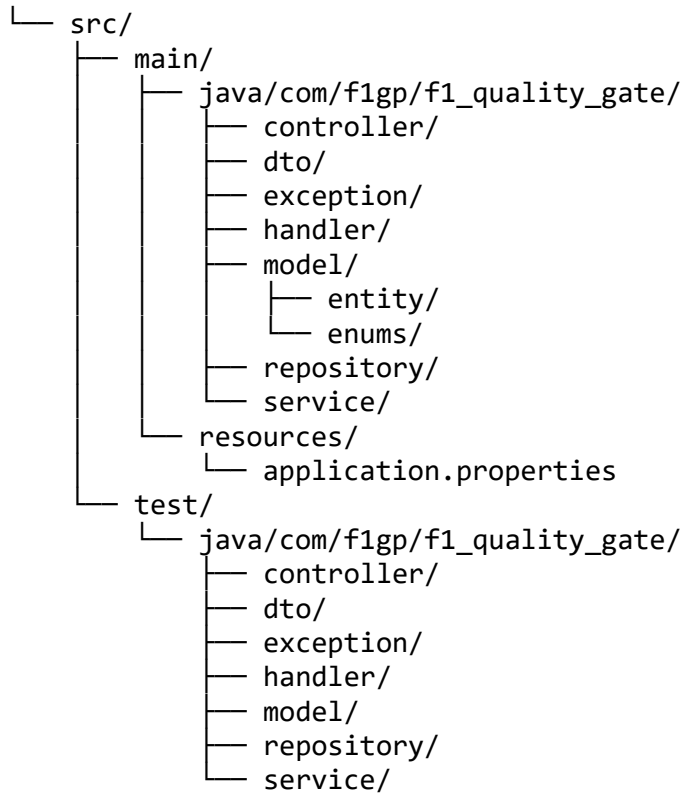
Table des matières

1 - Architecture du projet	1
1.1 - Architecture en couche	1
1.2 - Entités principales	1
1.3 - Justification du découpage.....	2
2 - Implémentation des règles métier	2
2.1 - Règles métier	2
2.2 - Endpoints implémentés	3
2.3 - Gestion des erreurs.....	3
3 - Métriques Sonar finale	4
3.1 - Dashboard final	4
3.2 - Tableau des métriques	4
3.3 - Analyse.....	4
4 - Tests Bruno (recette)	4
5 - Conclusion	7

1 - Architecture du projet

1.1 - Architecture en couche

Le projet est découpé en plusieurs couches afin de séparer les responsabilités.



Couche	Rôle
controller	Expose les endpoints REST
dto	Définit les objets d'entrée et de sortie de l'API
service	Contient les règles métiers
repository	Gère l'accès aux données avec Spring Data JPA
model/entity	Contient les entités persistées en base
model/enums	Contient les énumérations métier
exception	Contient les exceptions applicatives
handler	Centralise la gestion des erreurs HTTP

1.2 - Entités principales

Entité	Description
Grandstand	Représente une tribune du Grand Prix
RaceSession	Représente une session du week-end : essais, qualifications, sprint ou course
Spectator	Représente un spectateur inscrit
Reservation	Représente une réservation confirmée ou annulée

1.3 - Justification du découpage

Le découpage en couches permet de garder un code lisible et testable.

Les controllers ne contiennent pas de logique métier. Ils reçoivent les requêtes HTTP, délèguent aux services, puis retournent les réponses.

La logique métier est placée dans les services :

Service	Rôle
PricingService	Calcul du prix d'une réservation
ReservationService	Création, disponibilité et annulation des réservations
RefundService	Calcul du remboursement
GrandstandService	Gestion des tribunes
RaceSessionService	Gestion des sessions
SpectatorService	Gestion des spectateurs

2 - Implémentation des règles métier

2.1 - Règles métier

Règle métier	Où dans le code	Tests associés
Calcul du prix de base : $\text{prix_tribune} \times \text{multiplicateur_session} \times \text{nb_places}$	PricingService	PricingServiceTest
Multiplicateurs par défaut : PRACTICE 0.5, QUALIFYING 1.0, SPRINT 1.2, RACE 1.8	RaceSessionService	RaceSessionServiceTest
Pass weekend : réduction de 20% si FRIDAY + SATURDAY + SUNDAY	PricingService	PricingServiceTest
Fidélité SILVER : -5%	PricingService	PricingServiceTest
Fidélité GOLD : -10%	PricingService	PricingServiceTest
Tarif jeune : -50% si le spectateur a moins de 16 ans	PricingService	PricingServiceTest
Limite de 6 places par réservation	DTO QuoteRequest, ReservationRequest avec Bean Validation	ReservationControllerTest, ReservationServiceTest
Disponibilité par session et tribune	ReservationService	ReservationServiceTest, recette Bruno
Email unique	SpectatorService, SpectatorRepository	SpectatorServiceTest, SpectatorRepositoryTest
Format email valide	SpectatorRequest	SpectatorControllerTest
Date de naissance dans le passé	SpectatorRequest	SpectatorControllerTest

Multiplicateur de session > 0	RaceSessionRequest, RaceSession	RaceSessionControllerTest
Annulation > 7 jours : remboursement 100%	RefundService, ReservationService	RefundServiceTest, ReservationServiceTest
Annulation < 7 jours : remboursement 0%	RefundService	RefundServiceTest
Réservation déjà annulée	ReservationService	ReservationServiceTest, ReservationControllerTest

2.2 - Endpoints implémentés

Fonctionnalité	Endpoint
Créer une tribune	POST /api/grandstands
Lister / filtrer les tribunes	GET /api/grandstands?category=GOLD
Créer une session	POST /api/sessions
Inscrire un spectateur	POST /api/spectators
Simuler le prix d'une réservation	POST /api/reservations/quote
Créer une réservation	POST /api/reservations
Annuler une réservation	POST /api/reservations/{id}/cancel

2.3 - Gestion des erreurs

Format

L'API retourne les erreurs au format :

```
{
  "error": "Message d'erreur"
}
```

Types

Code HTTP	Cas
400 Bad Request	Données invalides
404 Not Found	Ressource introuvable
409 Conflict	Conflit métier

Exemple de conflits métier

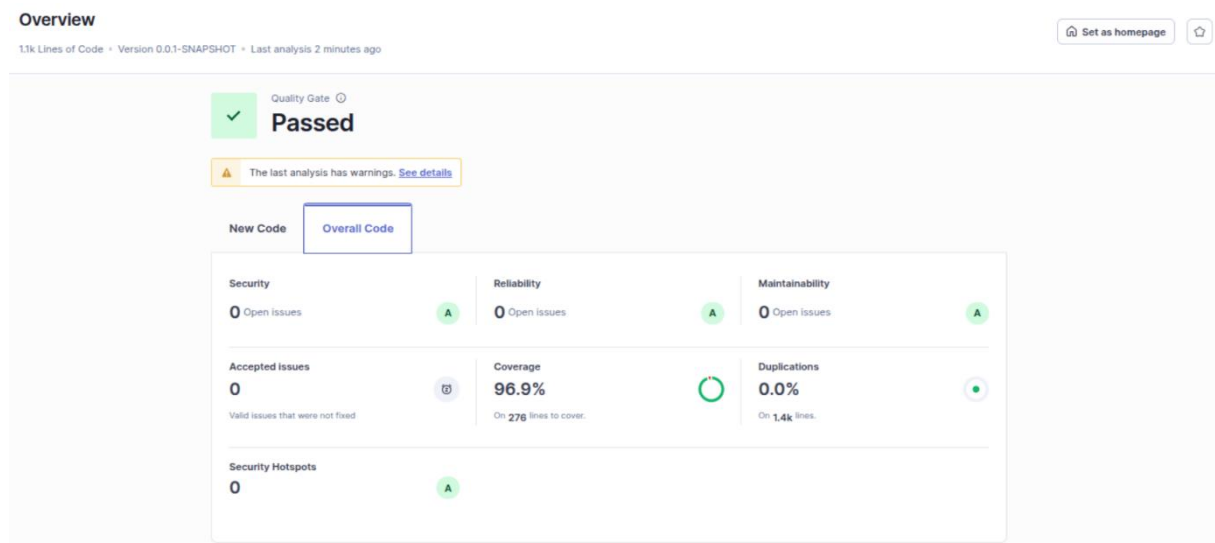
```
{
  "error": "Email déjà utilisé"
}

{
  "error": "Pas assez de places sur la session SUNDAY – RACE"
}

{
  "error": "Réservation déjà annulée"
}
```

3 - Métriques Sonar finale

3.1 - Dashboard final



3.2 - Tableau des métriques

Métrique	Valeur finale
LOC	1100
Security	0
Coverage	96.9%
Duplications	0.0%
Security Hotspots	0
Issues	0
Quality Gate	Passed

3.3 - Analyse

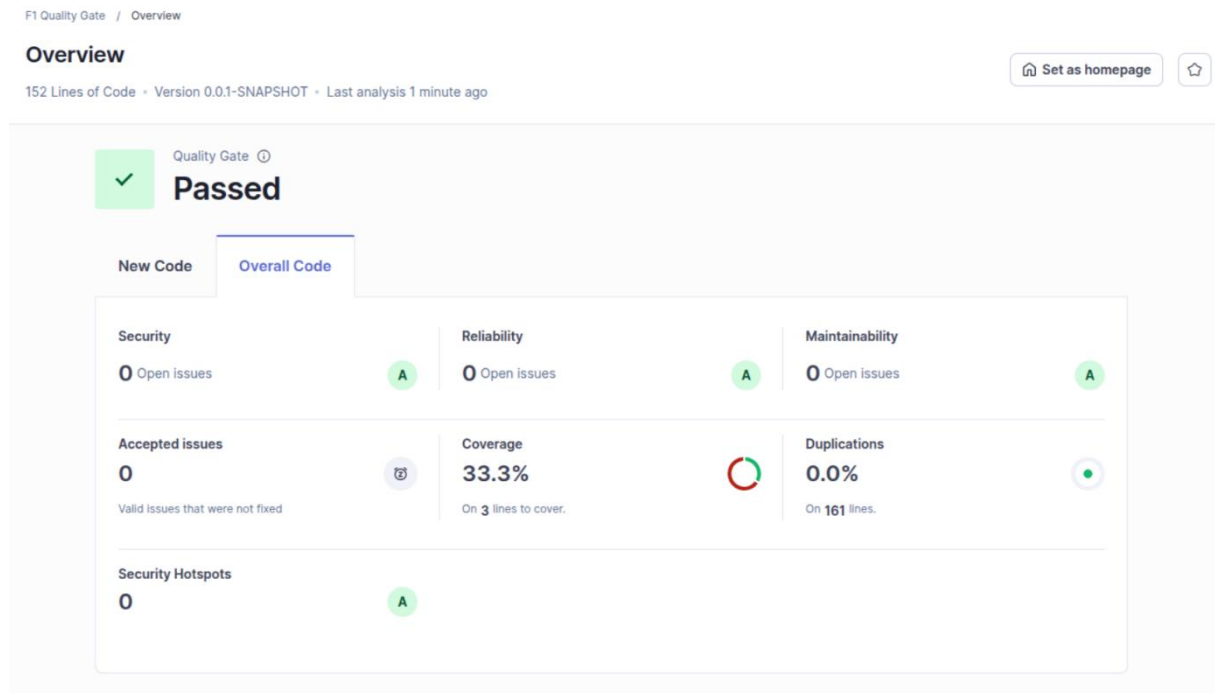
Métrique	Condition attendue	Résultat
New Issues	0	Ok
Security Hotspots Reviewed	100%	Ok
Coverage on New Code	≥ 4%	Ok
Duplicated Lines on New Code	≤ 3%	Ok

Le projet a été testé au-delà du minimum attendu. Les tests couvrent les services métier, les controllers, les DTOs, les repositories et les exceptions.

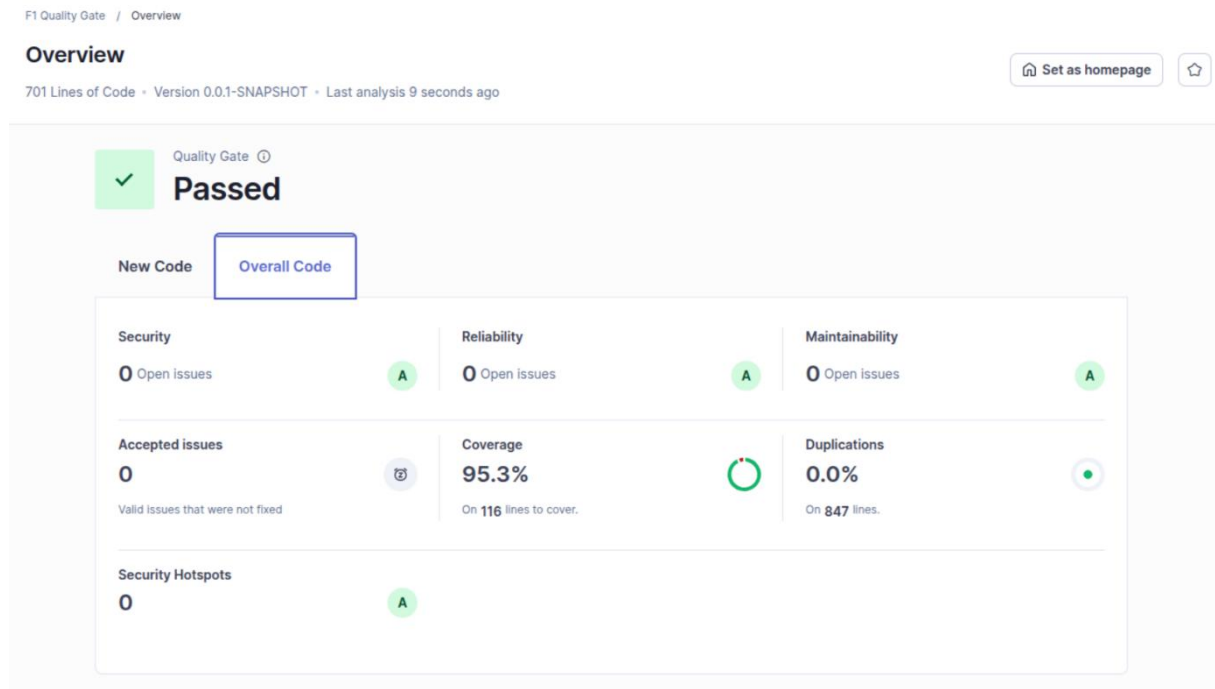
4 - Historique de qualité

Voici 3 captures de Sonar au début, au milieu et à la fin du projet.

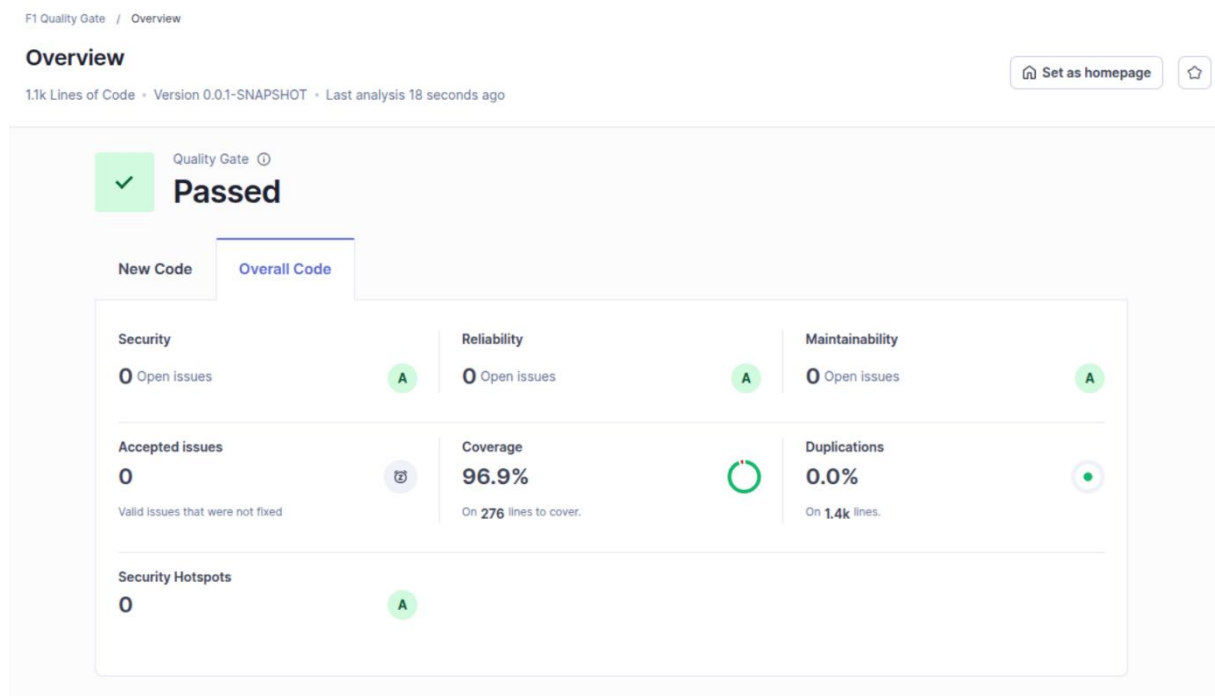
4.1- Début (commit : *d85bfec*)



4.2 - Milieu (commit : *29a8917*)



4.3 – Fin (commit : 29a8917)



5 - Tests Bruno (recette)

La totalité du projet est testé via Bruno. Les collections et environnements sont disponible dans le `./bruno` du projet.

La recette confirme que :

- Les endpoints fonctionnent ;
- Les règles métier sont appliquées ;
- Les erreurs sont correctement gérées ;
- Le format JSON des erreurs est homogène ;
- Les cas nominaux et les cas d'échec sont couverts.

6 - Bilan critique

6.1 - Difficultés rencontrées

Plusieurs difficultés ont été rencontrées pendant le projet.

Une difficulté concernait SonarQube. Après une le redémarrage de Sonar, la base Sonar devait être migrée manuellement via /setup. Tant que cette étape n'était pas faite, Sonar restait en maintenance.

Des erreurs de port déjà utilisé ont également été rencontrées lors du lancement de Spring Boot.

Les tests repository ont posé un problème car ils utilisaient la base MariaDB contenant déjà des données de recette Bruno. Les tests n'étaient donc pas totalement isolés. La solution utilisée a été de nettoyer les données avant les tests. Une meilleure solution serait d'utiliser une base de test dédiée ou Testcontainers.

6.2 - Règles Sonar contestables

Certaines règles Sonar sont utiles mais peuvent être discutées dans un contexte scolaire.

Par exemple, Sonar signale parfois les lambdas dans les tests d'exception lorsqu'elles contiennent plusieurs appels. La règle est pertinente pour éviter les tests ambigus, mais elle peut être assez stricte sur du code de test simple.

Sonar peut également remonter des alertes sur du code généré ou très simple, comme les entités JPA ou les DTOs. Dans le projet, les entités ont été exclues du calcul de couverture car elles contiennent principalement des champs, annotations et méthodes générées par Lombok. La logique métier réelle est dans les services, qui sont testés directement.

6.3 - Ce qui serait refait autrement

Avec plus de temps, plusieurs améliorations seraient possibles.

La première amélioration serait d'utiliser Testcontainers pour les tests repository. Cela permettrait de lancer une base MariaDB propre uniquement pour les tests, sans dépendre de la base utilisée pendant la recette manuelle.

Une autre amélioration serait d'ajouter une couche de mappers dédiée pour convertir les entités en DTOs. Actuellement, les conversions sont directement dans les services, mais une couche dédiée serait plus propre si le projet grossissait (record pour les I/O, pojo les manipulations et dto pour l'envoi en base de données).

Enfin, une CI GitHub Actions pourrait être ajoutée pour exécuter automatiquement :

- Les tests ;
- Le rapport JaCoCo ;
- L'analyse Sonar ;

- Le contrôle du format des commits.

6.4 - Conclusion

Le projet répond aux fonctionnalités obligatoires et facultatives ainsi qu'aux règles métier demandées.

La mise en place de SonarQube dès le début du projet a permis de suivre la qualité du code. Le projet respecte le Quality Gate finale et dispose de tests automatisés ainsi que d'une recette manuelle complète.

Le découpage en couches permet de maintenir une séparation claire entre l'API, la logique métier et l'accès aux données.